

Esercitazione regole di binding

1 Spiegare dove e perchè nel programma è necessario parlare di regole di scoping e dove (e perchè) anche di regole di binding. Si dica quale risulta essere il valore finale di c nelle varie situazioni, ovvero con:

1. scoping statico
 2. scoping dinamico e deep-binding
 3. scoping dinamico e shallow-binding
- definendo chiaramente i diversi ambienti usati nei vari casi

```
{
  int x = 4;
  int y = 6;

  int p(int a) {
    return x + a;
  }

  int q(int f(int b)) {
    int x = 3;
    return f(x) + x;
  }

  int exec () {
    int x = 2;
    return q(p);
  }

  int c = exec();
}
```

] 1 Definizione

] 3 esecuzione

] 2 Creazione del legame

Bisogna specificare:

- Quando si usano le regole di scoping (e le si definiscano se non fatto in esercizi precedenti).
Il punto chiave è: ambiente non locale in procedure
- Quando si usano le regole di binding (e le si definiscano se non fatto in esercizi precedenti).
Il punto chiave è: ambiente non locale in procedure passate come parametro ad altre procedure
- Definire le regole di binding:
 - * Scoping statico → ambiente della definizione
 - * Scoping dinamico + deep binding → ambiente della creazione del legame tra parametro formale e attuale della funzione passata
 - * Scoping dinamico + shallow binding → ambiente della chiamata del parametro attuale

- in p → x non locale, a locale
- in q → x locale

Scoping statico

Si utilizza l'ambiente della definizione $1 \rightarrow x=4$

$q(p) \rightarrow F(x)+x \rightarrow x+a$
 _{locl 3 + 3} _{4 locale 3}
 _{amb 1}
 $c=10 \leftarrow 7$

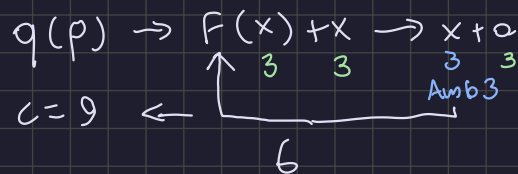
Scoping dinamico + deep binding

Si utilizza l'ambiente alla creazione del legame $2 \rightarrow x=2$

$q(p) \rightarrow F(x)+x \rightarrow x+a$
 _{3 3} _{2 3}
 _{Amb 2}
 $c=8 \leftarrow 5$

Scoping dinamico + shallow binding

Si utilizza l'ambiente all'esecuzione $3 \rightarrow x = 3$



```
2 {
  int x = 4;
  int y = 6; ]1 Definizione (statico per tutti)

  int p(int a) {
    return x + y - a;
  }

  int q(int f(int b)) {
    int x = 3; ]3 Chiamata di F (dinamico per F)
    return f(x) + y;
  }

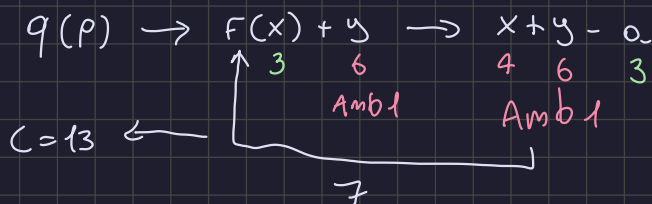
  int exec() {
    int y = 5; ] Creazione del legame (dinamico chiamato di q)
    return q(p);
  }

  int c = exec();
}
```

- per $p \rightarrow x$ non locale, y non locale
- per $q \rightarrow x$ locale, y non locale, siccome q non è mai passata per parametro bastano le regole di scoping

Scoping statico

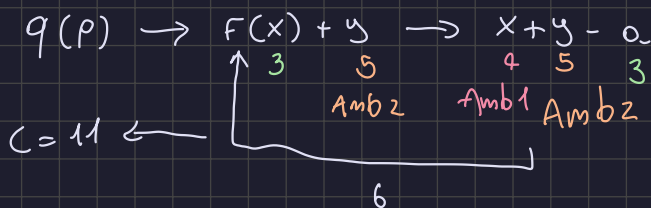
Si utilizza l'ambiente di definizione $1 \rightarrow x = 4, y = 6$



Scoping dinamico + deep binding

Per p (ovvero f), l'ambiente di riferimento è quello alla creazione del legame $2 \rightarrow x = 4, y = 5$

Per q , l'ambiente di riferimento è quello dinamico per le regole di scoping: $2 \rightarrow x = 4, y = 5$

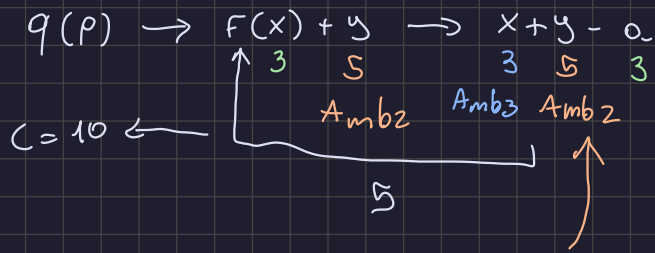


Siccome l'ambiente 2 non definisce x si deve risalire all'ambiente genitore (del chiamante) che è l'ambiente 1

Scoping dinamico + shallow binding

Per p (cioè f) l'ambiente è quello di esecuzione: 3

Per q, l'ambiente è quello dinamico per le regole di scoping: 2



La regola di visibilità per y dice che l'ambiente precede l'ambiente 3

3

{ 1. Ambiente di definizione delle procedure (scoping statico)

```
int x = 4;
int z = 0;
```

```
int f(int y) {
  return x * y // x ambiente non locale
}
```

Solo regole di scoping

```
void g(int h(int n)) {
  int x = 7;
  int z = h(3) + x;
}
```

{ 3. Ambiente della chiamata di f (attraverso h) scoping dinamico + shallow binding

Tutto locale

```
{ 2. Ambiente in cui f viene passato come parametro (della creazione del legame con h) scoping dinamico +
  int x = 5;
  g(f);
}
```

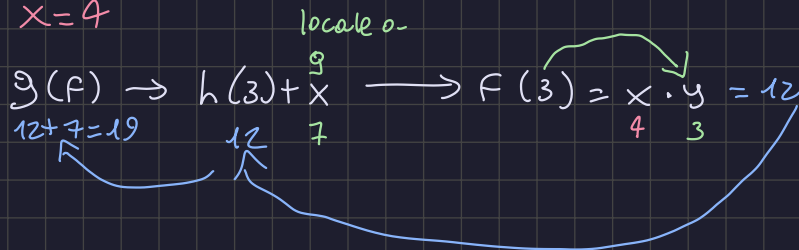
scoping + binding

depp binding

Trovare il valore di z nei tre casi:

Scoping statico (Ambiente 1.)

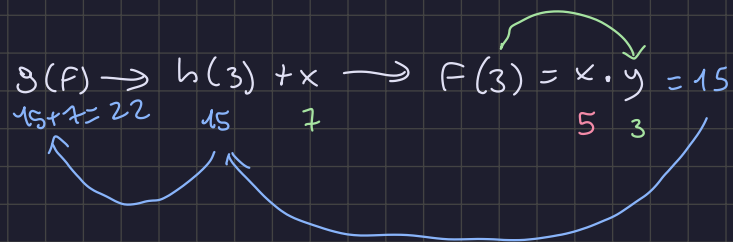
x=4



z=19

Scoping dinamico + deep binding (Ambiente 2)

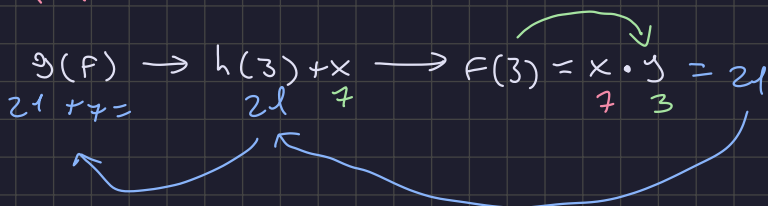
x=5



z=22

Scoping dinamico con shallow binding (Ambiente 3)

x=7



z=28

4 { 1. Ambiente di definizione di f e di g, scoping statico

```
int x = 3;
int y = 2;
```

```
int f(int n) {
  return x + y - n;  x ↗ non locale
}
```

int g(int h(int b)) { 3. Ambiente di chiamata di f, scoping dinamico + shallow binding

```
int x = 4;
return h(5) + y;  y non locale
}
```

{ 2. Ambiente di chiamata di g (scoping dinamico) e di passaggio di parametro per f (scoping dinamico + deep binding)

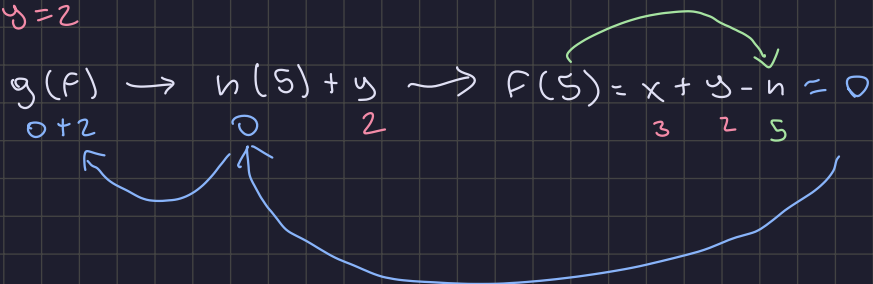
```
int x = 6;
int y = 3;
int z = g(f);  scoping + binding
}
↓
regole di scoping
```

Trovare il valore di z nei tre casi:

Scoping statico (Ambiente 1)

x=3

y=2

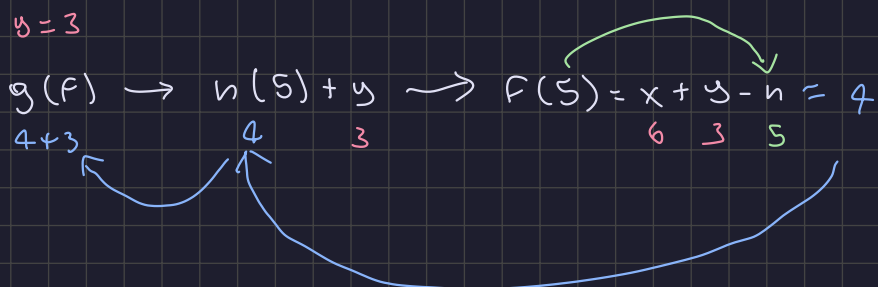


z=2

Scoping dinamico con deep binding (Ambiente 2)

x=6

y=3

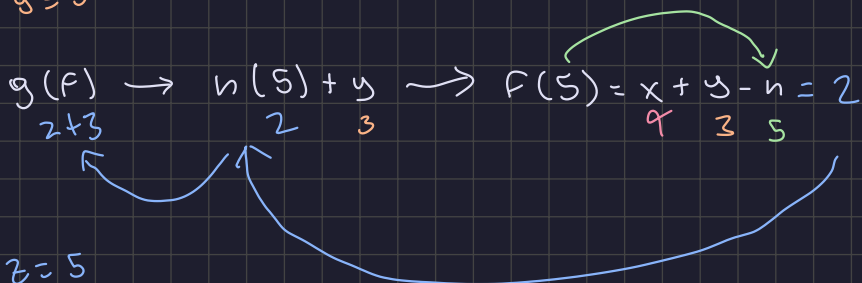


z=7

Scoping dinamico + shallow binding (Ambiente 2 e 3)

x=4

y=3



z=5